2.1. Contexto del problema

• Nombre del proyecto

UniProject: sistema de gestión de proyectos colaborativos universitarios.

• Descripción clara del proyecto

En los cursos universitarios, buena parte de la nota depende de proyectos grupales que se entregan por hitos durante el semestre. Es frecuente que la carga termine en manos de unos pocos integrantes y que una tarea atrasada frene a las que dependen de ella sin que nadie lo advierta hasta que la entrega está encima. El docente, por su lado, recibe el producto terminado sin saber cómo se repartió el trabajo. Así que califica a todos por igual.

El docente crea el proyecto en UniProject y define sus hitos, cada uno con fecha límite y rúbrica de evaluación. Después, el líder de cada equipo divide el trabajo en tareas y las asigna a sus compañeros. Al crear cada tarea estima cuánto va a durar e indica de qué otras tareas depende. Con esa información el sistema calcula la ruta crítica del proyecto, y si una tarea de esa ruta se atrasa hasta poner en riesgo la fecha del hito, avisa al equipo y al docente.

Cada estudiante registra en sus tareas el porcentaje de avance y adjunta las evidencias de su trabajo (archivos PDF, imágenes o enlaces a repositorios). El líder revisa esas evidencias y, si encuentra errores, puede pedirle al integrante que rehaga la entrega indicando qué debe corregir. Mientras haya tareas marcadas como prerrequisito crítico sin terminar, el equipo no puede enviar el hito a revisión. Una vez enviado, el docente lo califica con la rúbrica. Si la fecha límite del hito no ha vencido, esa nota es provisional y el equipo puede seguir editando sus tareas; cuando la fecha vence, la última nota asignada queda como definitiva. A partir de la calificación grupal, el sistema le sugiere al docente una nota individual para cada integrante, ajustada según el peso de las tareas que cada estudiante completó a tiempo. La nota final la decide el docente.

El docente puede configurar alertas de inactividad, que se disparan cuando un estudiante pasa el tiempo que él defina sin registrar actividad en sus tareas. La alerta siempre le llega al líder del equipo, y el docente decide si también quiere recibirla. Si la institución trabaja con monitores, el docente puede asignarlos al proyecto para que comenten las tareas en curso sin cambiar su estado ni su avance. Las reasignaciones de tareas quedan en un registro de auditoría que nadie puede modificar, y al final del semestre se puede exportar en PDF un reporte con el historial completo del equipo.

Herramientas como Trello o Asana sirven para organizar pendientes, pero no manejan hitos con rúbrica, no bloquean una entrega por dependencias sin cumplir y no convierten el aporte de cada persona en una nota. UniProject está construido alrededor de ese ciclo académico.

• Alcance de la solución: Objetivo General y Objetivos Específicos

Objetivo general

Diseñar y construir un sistema web para gestionar proyectos académicos grupales organizados por hitos, que controle las dependencias entre tareas, detecte a tiempo los retrasos que comprometen las entregas y mida la contribución individual de cada integrante, de modo que el trabajo se reparta de forma más equitativa y el docente pueda seguir el proceso y no solo el resultado.

Objetivos específicos

1. Permitir al docente crear proyectos académicos con varios hitos de entrega, cada uno con su fecha límite y su rúbrica de evaluación.
2. Permitir al líder del equipo crear tareas, asignarlas entre los integrantes, definir dependencias entre ellas y reasignarlas cuando haga falta.
3. Calcular y graficar de forma automática la ruta crítica del proyecto a partir de las dependencias y los tiempos estimados, y recalcularla cada vez que estos cambien.
4. Registrar el avance de cada tarea (porcentaje de avance y evidencias), detectar los retrasos en tareas críticas con notificación al equipo y al docente, y generar alertas para el líder del equipo, y de forma opcional para el docente, cuando un estudiante no registre actividad durante el periodo definido.
5. Controlar el flujo de entrega y calificación de cada hito, desde el bloqueo del envío cuando hay prerrequisitos críticos incompletos hasta la nota del docente, que es provisional mientras la fecha límite del hito no haya vencido.
6. Calcular y sugerir al docente la nota individual de cada estudiante a partir de la calificación grupal del hito, con un factor de ajuste basado en el peso de las tareas que completó a tiempo.
7. Restringir las acciones disponibles según el rol de cada usuario y mantener la trazabilidad del proyecto mediante un registro de auditoría, un panel de control para el docente y un reporte exportable en PDF.

Quedan fuera del alcance de esta versión la integración con plataformas institucionales, el chat y las videollamadas entre integrantes, las aplicaciones móviles nativas y la evaluación automática del contenido de los entregables, que siempre califica el docente.

• Supuestos

1. Todos los usuarios tienen acceso a internet y a un dispositivo con un navegador web actualizado.
2. Cada equipo tiene un único líder, designado cuando se conforma el equipo.
3. El docente define los hitos, las rúbricas y el periodo de inactividad que dispara las alertas antes de que el equipo empiece a trabajar en cada hito.
4. Los estudiantes registran su porcentaje de avance con honestidad y a tiempo. El sistema guarda lo que se reporta.
5. Los tiempos estimados de las tareas los fija el líder, así que tanto la ruta crítica como el ajuste de la nota individual serán tan precisos como lo sean esas estimaciones.
6. El semestre académico dura 16 semanas y el uso de la plataforma sube en las semanas 8 y 16, que coinciden con las entregas principales.
7. Los entregables pesados (código, bases de datos, videos) se guardan en servicios externos, como repositorios o almacenamiento en la nube, y en el sistema se adjuntan como enlaces.

• Restricciones

1. Por ser un proyecto académico, el desarrollo está limitado a la duración del semestre, lo hace un equipo de estudiantes y no cuenta con presupuesto, por lo que se usan herramientas gratuitas o con licencia académica.
2. La solución es una aplicación web que se usa desde el navegador. No se desarrollarán aplicaciones móviles nativas.
3. Las contraseñas se almacenan con bcrypt y un factor de costo mínimo de 12.
4. El sistema debe hacer cumplir estas reglas de negocio:
   a. Un estudiante regular no puede modificar el estado ni los adjuntos de una tarea asignada a otro compañero.
   b. Si el proyecto tiene monitores, estos pueden comentar las tareas, pero no pueden cambiar su estado ni su porcentaje de avance.
   c. Un hito no se puede enviar a revisión si tiene tareas marcadas como prerrequisito crítico en estado incompleto.
   d. Las dependencias entre tareas no pueden formar ciclos.
   e. El registro de auditoría no se puede editar ni borrar.

5. Las evidencias de una tarea solo pueden ser archivos PDF, imágenes o enlaces.
6. El manejo de los datos personales de estudiantes y docentes debe cumplir la Ley 1581 de 2012 de protección de datos personales en Colombia.

• Justificación de la propuesta

Cualquiera que haya hecho trabajos en grupo en la universidad ha visto al compañero que casi no aporta y recibe la misma nota que los demás. El docente tampoco tiene cómo notarlo, porque le llega un entregable terminado y muy poca información sobre cómo se hizo. Las herramientas comerciales de gestión de tareas ayudan a llevar pendientes, pero no están pensadas para hitos con rúbrica ni para calcular notas individuales.

UniProject responde a eso desde el diseño. Las alertas de retraso sobre la ruta crítica dan margen para reaccionar antes de que la fecha límite esté perdida, mientras que el registro de avances, evidencias y cambios le permite al docente ver cómo avanzó cada equipo. Y la nota individual sugerida, que ajusta la calificación grupal según las tareas que cada estudiante cumplió a tiempo, le da al docente una base concreta para no calificar a todos por igual.

2.2. Actores del sistema

• Identificación de actores

1. Docente (actor principal). Es el profesor del curso. Crea y configura los proyectos, supervisa el trabajo de los equipos y evalúa las entregas.
2. Estudiante (actor principal). Es cada integrante de un equipo de proyecto. Ejecuta las tareas que se le asignan y reporta su avance.
3. Líder de equipo (actor principal). Es un estudiante con permisos adicionales para organizar el trabajo de su equipo. Se modela como una especialización del actor Estudiante, así que conserva todas las responsabilidades de este y suma las propias.
4. Monitor (actor principal y opcional). Solo participa si la institución trabaja con monitores; en ese caso, el docente lo asigna al configurar el proyecto. Acompaña el trabajo de los equipos con retroalimentación, sin intervenir en el estado de las tareas.
5. Sistema de notificaciones (actor secundario). Es el sistema externo que hace llegar a los usuarios los avisos que genera UniProject.

• Responsabilidades

Docente

1. Crear proyectos académicos y definir sus hitos, cada uno con fecha límite y rúbrica de evaluación.
2. Configurar las alertas de inactividad, el periodo sin actividad que las dispara y si él también quiere recibirlas.
3. Supervisar a los equipos desde el panel de control, donde se resaltan los equipos inactivos y los que tienen tareas bloqueadas por más de 48 horas.
4. Calificar con la rúbrica los entregables de cada hito enviado. Mientras la fecha límite no haya vencido, la nota es provisional y el equipo puede seguir editando; al vencer, queda como definitiva.
5. Revisar la nota individual que sugiere el sistema para cada estudiante y asignar la nota final.
6. Consultar y exportar el reporte de trazabilidad del equipo en PDF.

Estudiante

1. Consultar las tareas que tiene asignadas y sus fechas límite.
2. Actualizar el porcentaje de avance de sus tareas.
3. Adjuntar a sus tareas las evidencias de su trabajo (archivos PDF, imágenes o enlaces a repositorios). Solo puede modificar el estado y los adjuntos de las tareas asignadas a él.
4. Atender las alertas de retraso y, si no puede terminar a tiempo, solicitar apoyo al líder.
5. Enviar el hito a revisión cuando todos los prerrequisitos críticos estén completos.
6. Consultar el resultado de la evaluación de cada hito.
7. Atender las solicitudes de corrección del líder y adjuntar la nueva versión de la entrega.

Líder de equipo (además de las responsabilidades del estudiante)

1. Dividir el trabajo del hito en tareas y estimar la duración de cada una.
2. Asignar las tareas a los integrantes según lo discutido internamente por el grupo
3. Establecer las dependencias entre tareas y marcar cuáles son prerrequisitos críticos del hito.
4. Analizar la carga del equipo cuando un integrante pide apoyo y reasignar tareas o parte de su alcance cuando haga falta. Cada reasignación queda en el registro de auditoría.
5. Revisar las evidencias que adjuntan los integrantes y, si encuentra errores, solicitar que rehagan la entrega indicando los puntos que deben corregir.

Monitor

1. Consultar las tareas en curso de los equipos.
2. Dejar comentarios de retroalimentación en esas tareas, sin cambiar su estado ni su porcentaje de avance.

Sistema de notificaciones

1. Avisar al equipo y al docente cuando una tarea de la ruta crítica se retrase hasta comprometer la fecha del hito.
2. Enviar las alertas de inactividad al líder del equipo y, si el docente lo eligió, también al docente.
3. Informar al docente cuando un equipo envía un hito a revisión y a los estudiantes cuando el hito ha sido evaluado.
4. Avisar a los integrantes afectados cuando cambian sus asignaciones por una reasignación.
5. Avisar al integrante cuando el líder le solicita rehacer una entrega.

2.3. Equipo del proyecto

Juan Benavides, (rol: integrante), rasputin2631

Samuel Lambertino, (rol: integrante), frahklp1

Santy Baza, (rol: integrante), akko98

2.4. Requerimientos

Requerimientos funcionales

1. El sistema debe permitir al docente crear proyectos académicos configurando múltiples hitos de entrega y vinculando rúbricas de evaluación a cada uno.
2. El sistema debe permitir al estudiante con rol de "Líder" crear y asignar tareas a los miembros del equipo, estableciendo dependencias jerárquicas (e.g., la tarea A bloquea la tarea B).
3. El sistema debe calcular y graficar automáticamente la ruta crítica del proyecto basándose en las dependencias y los tiempos estimados de cada tarea.
4. El sistema debe enviar notificaciones automáticas al equipo y al docente cuando una tarea de la ruta crítica sufra un retraso que comprometa la fecha límite del hito.
5. El sistema debe permitir al líder del equipo revisar las evidencias que un integrante adjuntó en una tarea y, si encuentra errores, solicitarle que rehaga la entrega indicando los puntos que debe corregir. La tarea vuelve al estado "En progreso" hasta que el integrante adjunte la nueva versión.

6. El sistema debe permitir al docente configurar alertas automáticas de inactividad que se disparen si un estudiante no registra actividad en sus tareas asignadas (cambios de estado o evidencias) durante un periodo de tiempo definido. La alerta siempre se envía al líder del equipo, y el docente decide si también la recibe.
7. El sistema debe bloquear la opción de enviar el hito a revisión del docente si existen tareas marcadas como "prerrequisito crítico" en estado incompleto.
8. El sistema debe permitir al docente calificar el hito enviado, asignando una nota y una retroalimentación a cada criterio de la rúbrica. Si la fecha límite del hito no ha vencido, la nota queda como provisional y los estudiantes pueden seguir editando sus tareas. Cuando la fecha vence, la última nota asignada queda como definitiva.
9. El sistema debe identificar tareas con alto riesgo de retraso a partir de su porcentaje de avance, tiempo restante, duración estimada y dependencias
10. El sistema debe generar un panel de control (dashboard) para el docente, resaltando en rojo los equipos inactivos o aquellos con tareas bloqueadas por más de 48 horas.
11. El sistema debe permitir la reasignación de tareas entre los miembros del equipo, generando automáticamente un registro inmutable (log) de auditoría del cambio.
12. El sistema debe permitir a los estudiantes adjuntar evidencias (archivos PDF, imágenes o enlaces de repositorios) y actualizar el porcentaje de avance en el detalle de cada tarea.
13. El sistema debe restringir los permisos de modo que un estudiante regular no pueda modificar el estado ni los adjuntos de una tarea asignada a otro compañero.
14. Si la institución trabaja con monitores, el sistema debe permitir al docente asignarlos a un proyecto. El monitor puede agregar comentarios de retroalimentación en las tareas en curso sin alterar su estado.
15. El sistema debe exportar un reporte de trazabilidad en PDF que consolide el historial de cambios, entregas y calificaciones del equipo durante todo el semestre.

Requerimientos no funcionales

1. Rendimiento: El motor de dependencias debe calcular y renderizar la ruta crítica de proyectos con hasta 150 tareas en un tiempo de respuesta máximo de 2 segundos en el cliente.
2. Concurrencia: La plataforma debe soportar a 500 usuarios simultáneos interactuando en los módulos de evaluación sin que el tiempo de respuesta del servidor supere los 3 segundos.
3. Disponibilidad: El sistema debe garantizar un uptime mínimo del 99.5% durante las semanas pico (semanas 8 y 16 del calendario académico).

4. Seguridad: Las contraseñas de todos los actores del sistema deben almacenarse utilizando el algoritmo de hash bcrypt con un factor de costo (work factor) mínimo de 12.
5. Usabilidad: El flujo de interfaz para la creación de una nueva tarea con dependencia debe requerir un máximo de 4 clics desde el panel principal del equipo.

2.5 Estructura del sistema

- **Capa de Presentación (Frontend):** Interfaz de usuario con la que interactúan docentes y estudiantes. Su responsabilidad es capturar interacciones, mostrar tableros de avance y renderizar visualmente dependencias y bloqueos.
- **Capa de Enrutamiento y Control (API Gateway / Controladores):** Punto de entrada de peticiones. Su responsabilidad es interceptar las solicitudes, verificar los tokens de autenticación/roles y enrutar la petición al servicio adecuado.
- **Capa de Lógica de Negocio (Servicios Core):** Motor central del sistema. Su responsabilidad es procesar los algoritmos pesados, calcular rutas críticas, validar que las tareas bloqueadas no se ejecuten y calcular matemáticamente el Índice de Contribución Efectiva.
- **Capa de Acceso a Datos (Repositorios/DAO):** Intermediario con la base de datos. Su responsabilidad es traducir los objetos de negocio a consultas SQL y garantizar las transacciones seguras (ACID) en la persistencia.
- **Gestor Asíncrono de Notificaciones:** Componente paralelo. Su responsabilidad es despachar correos electrónicos y alertas internas sin detener el tiempo de respuesta de la interfaz del usuario.
- **Relación entre componentes:** La **Capa de Presentación** se comunica exclusivamente mediante peticiones HTTP con la **Capa de Enrutamiento y Control**. Esta delega la ejecución de reglas a la **Capa de Lógica de Negocio**, la cual se apoya en la **Capa de Acceso a Datos** para leer/escribir información. Si un proceso crítico finaliza (ej. retraso de tarea crítica), la Lógica de Negocio dispara un evento al **Gestor Asíncrono de Notificaciones**.

2.6 División por módulos

- **Módulo de Configuración Académica:**

- Funcionalidad: Permite a los docentes parametrizar el entorno de trabajo, instanciar proyectos, definir el cronograma de hitos obligatorios e integrar las rúbricas de evaluación base para cada entrega.
- **Módulo de Ejecución y Dependencias:**
- Funcionalidad: Centraliza el trabajo de los estudiantes. Gestiona la creación y asignación de tareas, controla los porcentajes de avance, permite al líder revisar las evidencias y pedir correcciones a los integrantes, y aplica las reglas restrictivas del motor de dependencias (bloqueando tareas cuyo prerrequisito no esté al 100 %).
- **Módulo de Evaluación y Calificación Algorítmica:**
- Funcionalidad: Congela la edición de tareas al enviar un hito, habilita la interfaz de corrección/retroalimentación para el docente y procesa el cálculo automático de notas individuales distribuyendo la calificación grupal según el peso del trabajo finalizado exitosamente por cada estudiante.
- **Módulo de Trazabilidad y Seguridad (Logs):**
- Funcionalidad: Opera silenciosamente registrando de forma inmutable (solo lectura) cada acción crítica del sistema: reasignaciones por cuellos de botella, solicitudes de ayuda, cambios de estado y control de accesos, sirviendo como fuente de datos verificable para el motor de calificaciones.

2.7. Artefactos obligatorios

Casos de uso

Nombre del caso de uso: Establecer dependencias

Actores involucrados: Líder

Descripción: El caso de uso permite al líder del grupo establecer relaciones de dependencias entre tareas que bloqueen otras hasta estar terminadas

Precondición: El estudiante líder debe tener la sesión iniciada, tener un proyecto asignado y deben existir al menos dos tareas

Flujo principal de eventos:

1. El líder entra al menú del proyecto y selecciona una tarea
2. El líder selecciona la opción de establecer dependencias
3. El líder selecciona la o las tareas prerrequisito
4. El sistema guarda la relación y recalcula la ruta crítica
5. Fin exitoso del caso de uso

Flujo alterno:

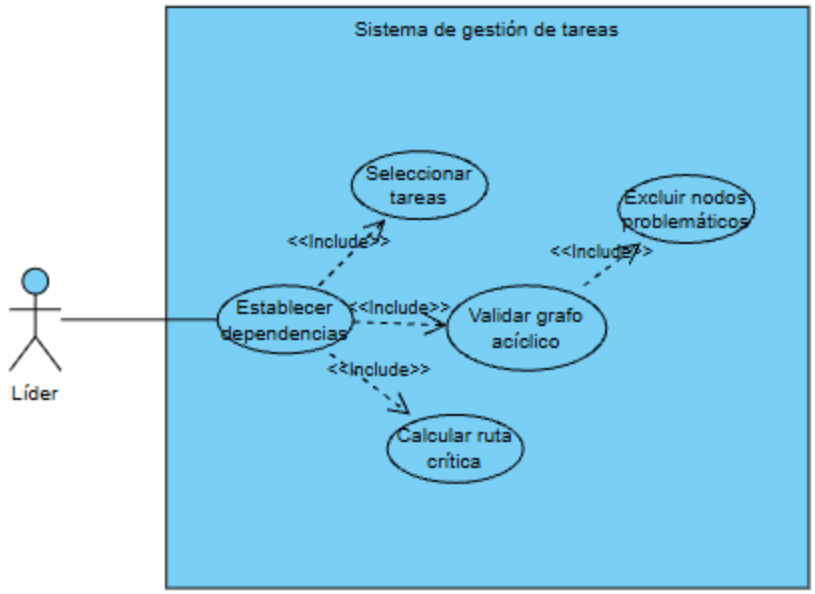
- Dependencia circular:

Si en el paso 3, alguna de las tareas o sus prerrequisitos recursivos dependen de la tarea actual:

- Se le notifica al líder la existencia de una dependencia circular
- Se excluye la o las tareas causantes
- El caso de uso continúa en el paso 4.

Postcondiciones:

Final exitoso: La tarea seleccionada inicialmente pasa a estar inhabilitada hasta que se completen sus dependencias



Nombre del caso de uso: Enviar hito

Actores involucrados: Estudiante, sistema de notificaciones

Descripción: El caso de uso detalla el proceso para enviar un hito del proyecto a revisión posterior por parte del docente

Precondición: El estudiante debe tener la sesión iniciada y debe haber al menos un hito disponible dentro de un proyecto que tenga

Flujo principal:

1. El estudiante selecciona la opción de enviar el hito a revisión
2. El sistema valida el estado de compleción de todos los prerrequisitos críticos
3. El sistema notifica mediante un diálogo el envío exitoso
4. El sistema notifica al docente
5. Fin exitoso del caso de uso

Flujos alternativos:

- Tareas críticas incompletas

Si en el paso 2. el sistema detecta que hay prerrequisitos críticos sin completar:

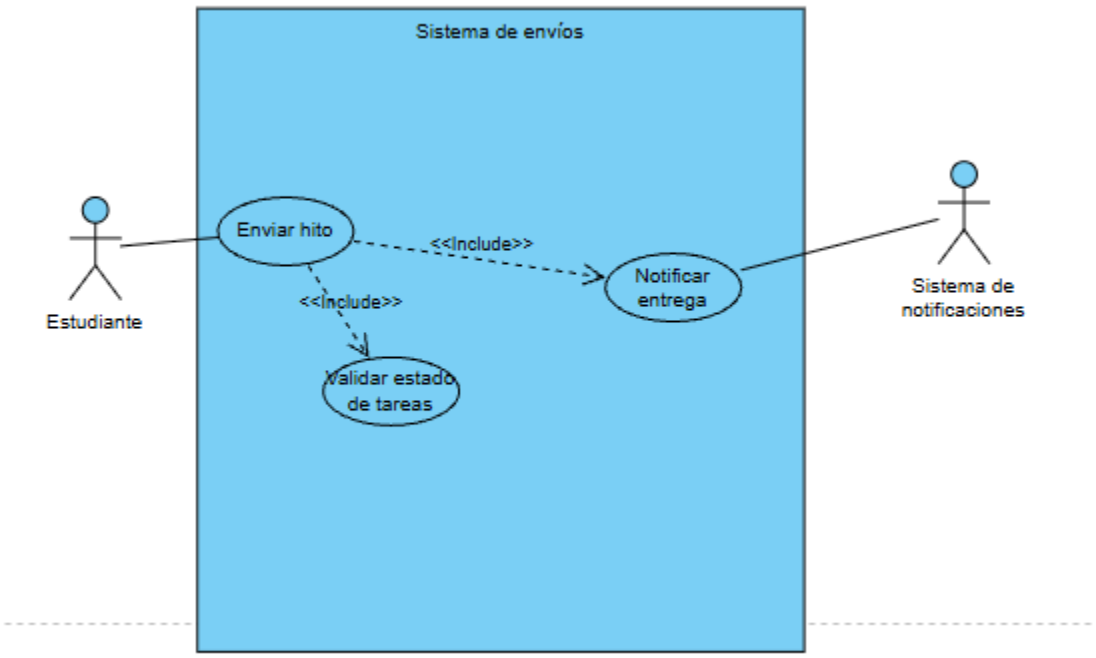
- Se le notifica al estudiante mediante un diálogo las tareas faltantes

- El caso de uso termina

Postcondiciones:

Final exitoso: Las entregas de las tareas del hito quedan bloqueadas, el docente tiene la entrega disponible en su bandeja de entrada

Condición de falla: No se realiza ningún envío y el hito se mantiene en estado editable



Nombre del caso de uso: Evaluar hito

Actores involucrados: Docente, sistema de notificaciones

Descripción: El caso de uso modela el proceso de evaluación de un hito por parte del docente y el registro de la nota y retroalimentación

Precondición: El docente debe haber iniciado sesión y tener en su bandeja de entrada al menos un hito entregado por calificar

Flujo principal:

1. El docente selecciona el hito por calificar
2. El docente revisa las entregas de las diferentes tareas
3. El docente asigna una nota y retroalimentación para cada elemento de la rúbrica

4. El sistema guarda las notas

5. El sistema permite la edición de los entregables

6. El sistema notifica a los estudiantes

7. Fin exitoso del caso de uso

Flujos alternativos:

- Fin del plazo

Si en el paso 5, el sistema halla que ya pasó la fecha límite de entrega

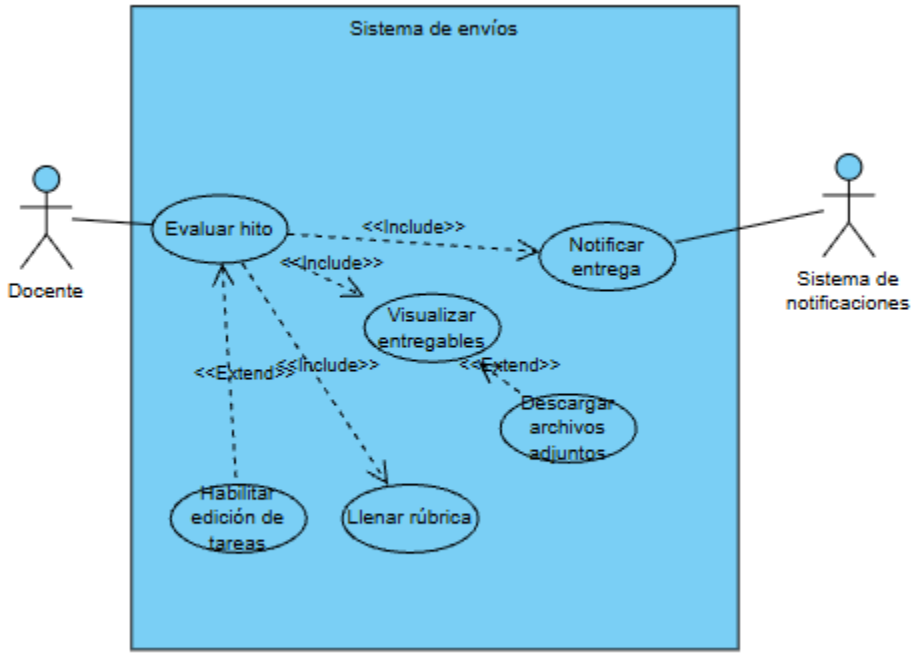
- No se habilita la edición posterior de entregables por los estudiantes

- El sistema registra la calificación como final

- Continúa el caso de uso en el paso 6

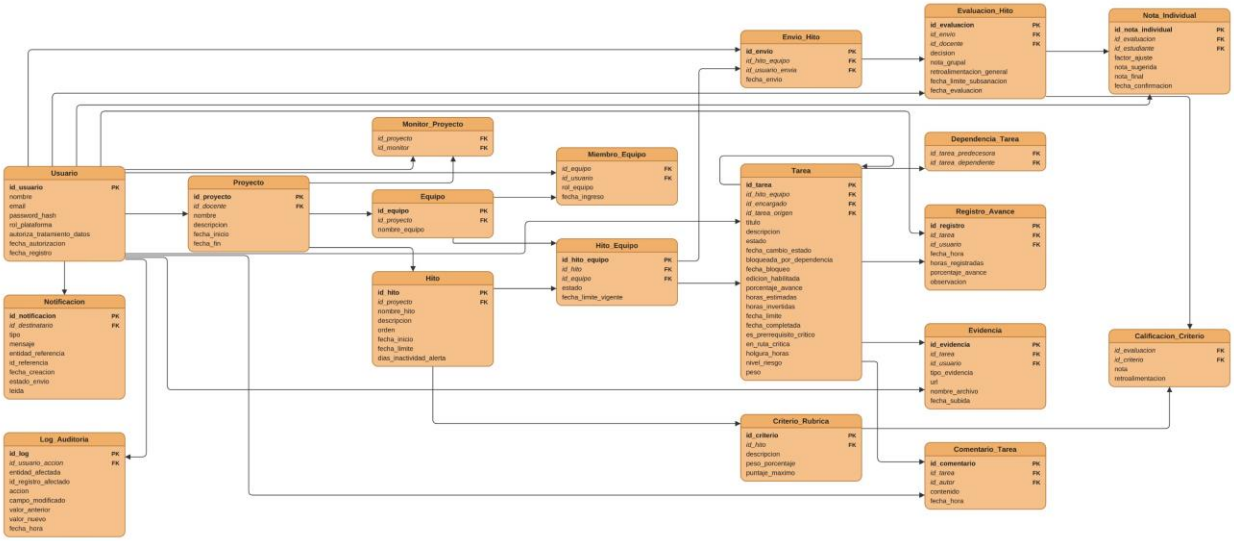
Postcondiciones:

Final exitoso: Los estudiantes fueron notificados del resultado temporal o final del hito



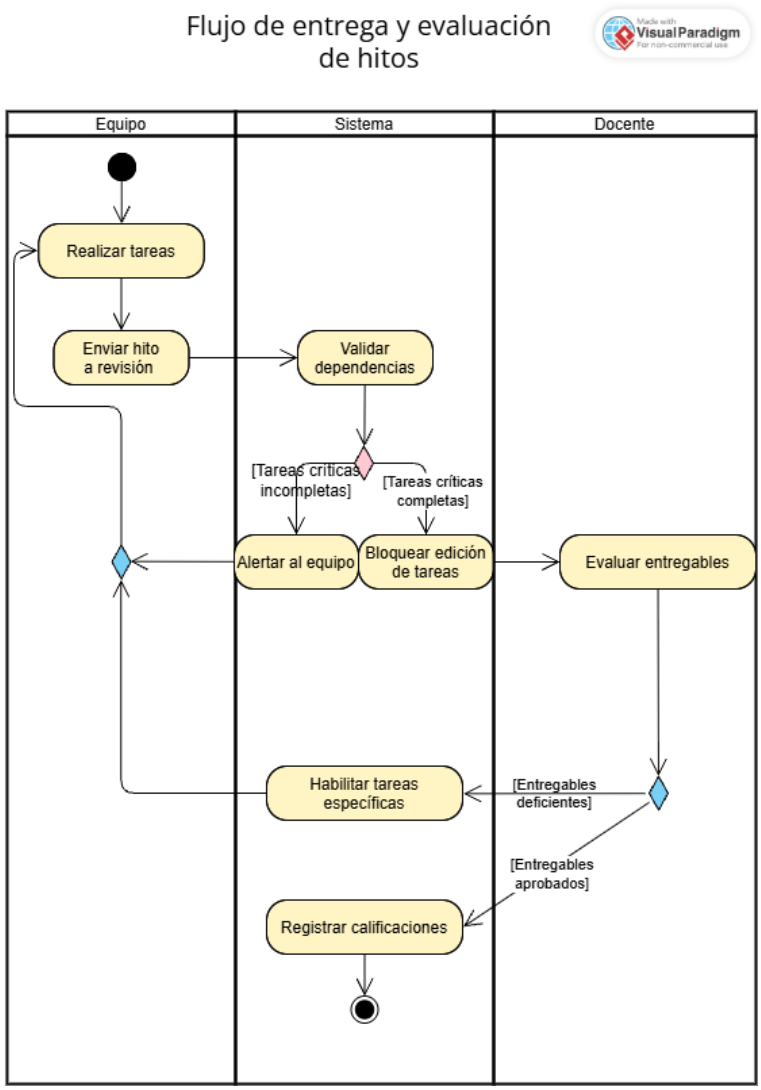
Prototipos

Diagrama de base de datos

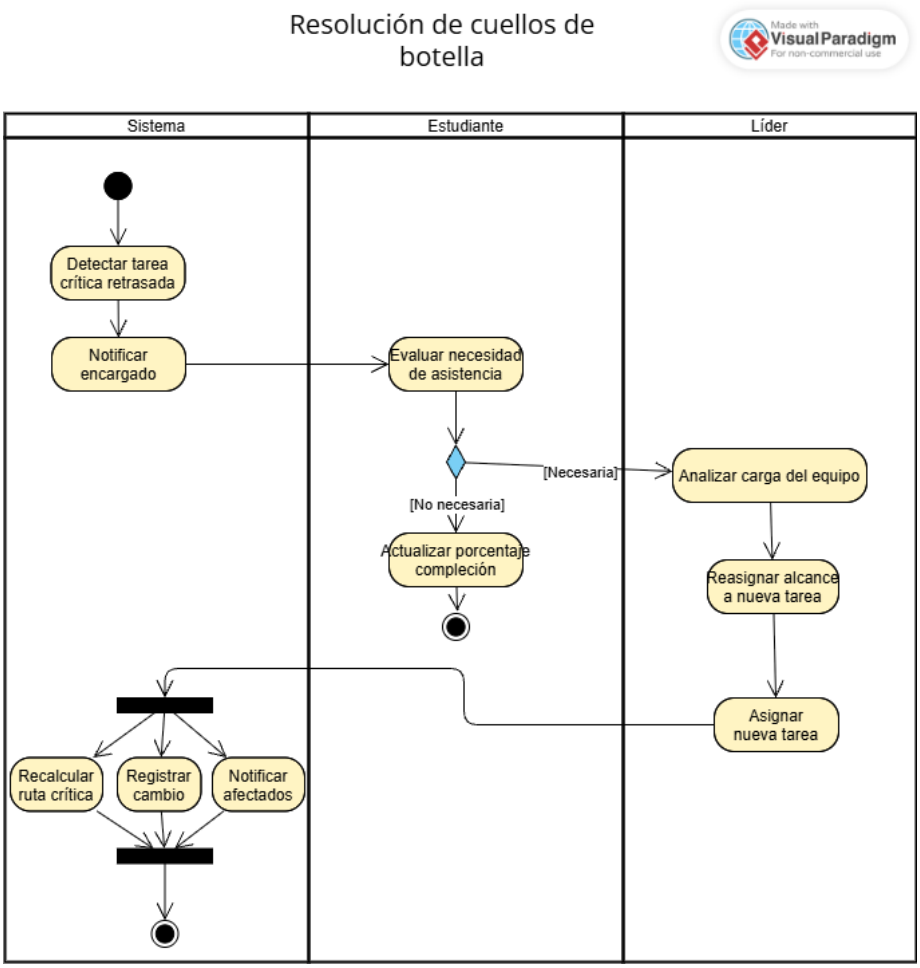


Diagramas de actividad

Flujo entrega y evaluación

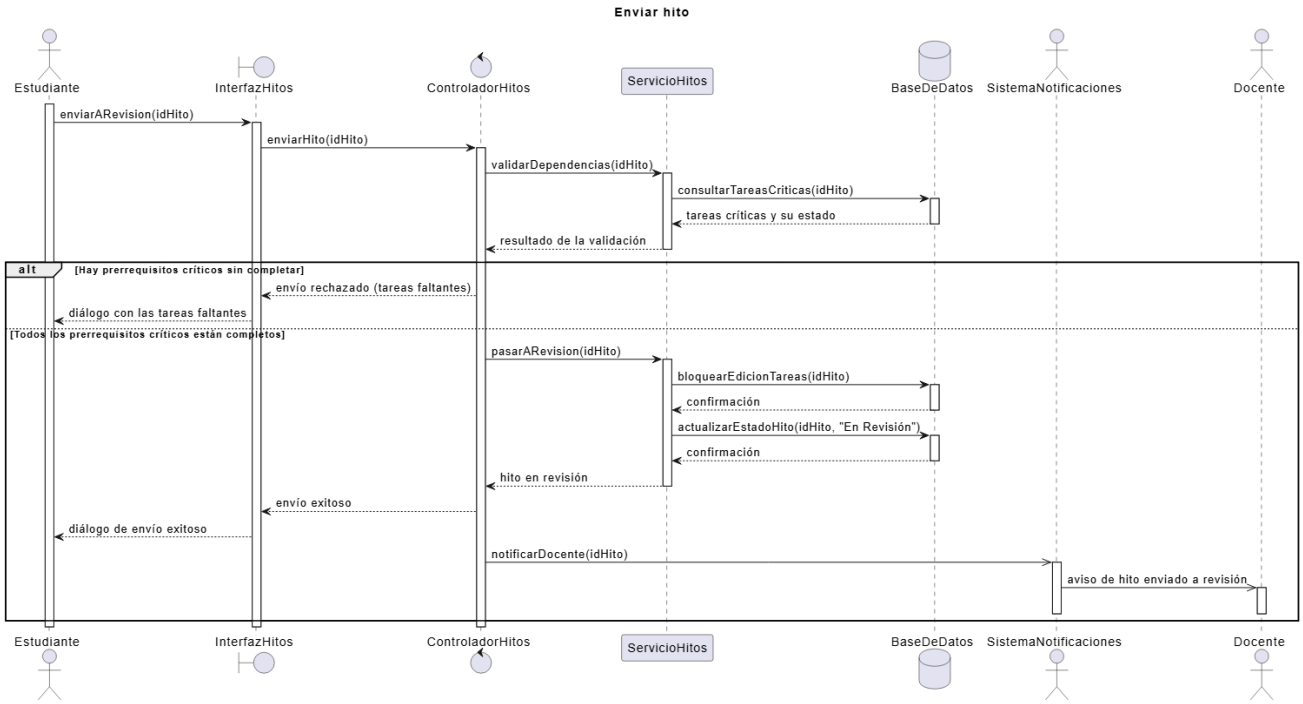


Resolución de cuellos de botella



Diagramas de secuencdia

Envío de hito para revisión académica



Establecer dependencias entre tareas

